School of Computation, Information and Technology
Technische Universität München

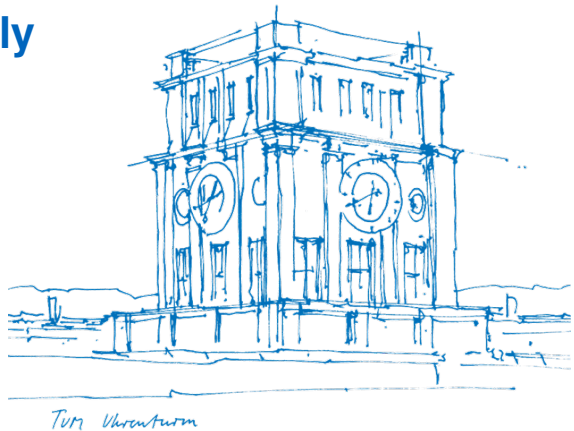
Übung 02: RISC-V Assembly

Einführung in die Rechnerarchitektur

Niklas Ladurner

School of Computation, Information and Technology
Technische Universität München

23. Oktober 2026



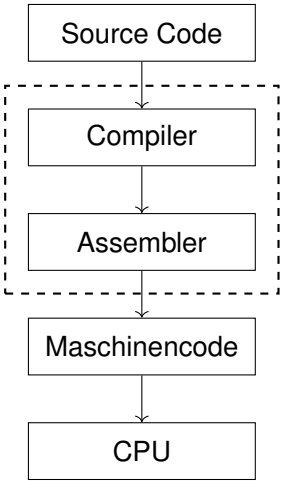
Keine Garantie für die Richtigkeit der Tutorfolien.

Bei Unklarheiten/Unstimmigkeiten haben VL/ZÜ-Folien recht!

Abstraktionsebenen

- Code in einer Hochsprache (C, Java[1], ...) ist lediglich eine Abstraktion
- Compiler: Hochsprache $\rightarrow$ Assemblercode
- Assembler: Assemblercode $\rightarrow$ Maschinencode (1:1 Übersetzung)
- Maschinencode ist plattformspezifisch!
- ISA: „Bedienungsanleitung“ einer CPU

[1] im Regelfall zu Bytecode kompiliert



(Abbildung stark vereinfacht)

RISC vs. CISC

Reduced **I**nstruction **S**et **C**omputer

- beschränkte Menge an Instruktionen
- einfache Implementierung, schnelle Dekodierung
- komplexere Operationen benötigen mehrere Instruktionen

Beispiel: RISC-V

Complex **I**nstruction **S**et **C**omputer

- umfangreiches Instruktionsset
- komplexe Implementierung, Realisierung als Mikrocode, langsame Dekodierung
- dedizierte Instruktionen auch für spezielle Operationen[2]

Beispiel: x86-64

[2] am Beispiel x86: `VCVTTPS2UQQ`, `GF2P8AFFINEINVQB`, `MASKMOVDQU` :)

RISC-V

- Befehlssatz in ERA: RV32IM
- 32 Register, einige davon mit spezieller Funktion
- Instruktionen auf 32 Bit begrenzt
  $\rightarrow$ Konstanten müssen zusammengebastelt werden
- Datenwortbreite: 32 Bit (4 Byte)
- Little-Endian-Architektur
- Byte-adressierbarer Speicher, maximale Hauptspeichergröße?



RISC-V

- Befehlssatz in ERA: RV32IM
- 32 Register, einige davon mit spezieller Funktion
- Instruktionen auf 32 Bit begrenzt
  $\rightarrow$ Konstanten müssen zusammengebastelt werden
- Datenwortbreite: 32 Bit (4 Byte)
- Little-Endian-Architektur
- Byte-adressierbarer Speicher, maximale Hauptspeichergröße?
  $\rightarrow 2^{32}$ Adressen, ca. $4{,}3$ GB



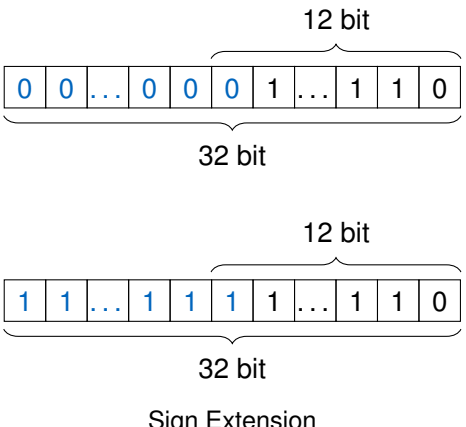
Register

Register	ABI Name	Description	Saver[3]
x0	zero	Hard-wired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5-x7	t0-t2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10-11	a0-1	Function arguments/return values	Caller
x12-17	a2-7	Function arguments	Caller
x18-27	s2-11	Saved registers	Callee
x28-31	t3-6	Temporaries	Caller

[3] Erst in zwei Wochen relevant

Immediates

- als „Konstanten“ in Instruktion enkodiert
- RV32IM verwendet 12- und 20-Bit Immediates
- Datenwortbreite 32 Bit $\rightarrow$ „Erweiterung“ von Immediates notwendig
- Sign- vs. Zero-Extension
- Achtung: 12-Bit-Immediates werden immer als signed Zahlen interpretiert!



Sign Extension

Instruktionsformat

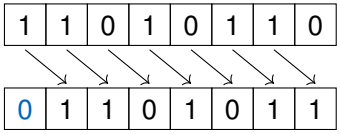
- Drei-Operanden-Form `op rd, rs1, rs2`
  (Operation, Zielregister, Quellregister 1, Quellregister 2)
- Register-Immediate-Operationen mit Suffix `i` (`addi`, `xori`, ...)
  $\rightarrow$ Konstante statt Quellregister 2
- Was ist mit `subi`?

Instruktionsformat

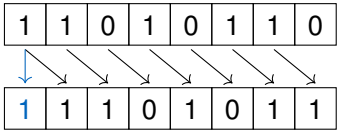
- Drei-Operanden-Form `op rd, rs1, rs2`
  (Operation, Zielregister, Quellregister 1, Quellregister 2)
- Register-Immediate-Operationen mit Suffix `i` (`addi`, `xori`, ...)
  $\rightarrow$ Konstante statt Quellregister 2
- Was ist mit `subi`?
  $\rightarrow$ existiert nicht, `addi` mit negativem Immediate

Schiebebefehle

- Logisch: Auffüllen mit 0
- Arithmetisch: Auffüllen mit Sign-Bit
- Shifts sind sehr hilfreich zum Dividieren bzw. Multiplizieren mit Zweierpotenzen
- Was ist mit `sla`?



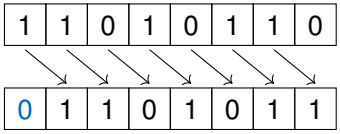
logischer Rechtsshift



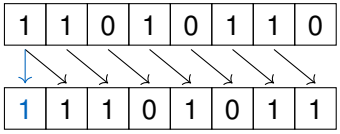
arithmetischer Rechtsshift

Schiebebefehle

- Logisch: Auffüllen mit 0
- Arithmetisch: Auffüllen mit Sign-Bit
- Shifts sind sehr hilfreich zum Dividieren bzw. Multiplizieren mit Zweierpotenzen
- Was ist mit `sla`?
  $\rightarrow$ existiert nicht, selber Effekt wie `sll`



logischer Rechtsshift



arithmetischer Rechtsshift

Fragen?

Links

- Zulip: „ERA Tutorium – Mi-1600-1“ bzw. „ERA Tutorium – Fr-1600-1“
- ERA-Moodle-Kurs
- ERA-Artemis-Kurs
- Einführung Fixpunktarithmetik, Alternative
- Übersicht an RISC-V-Instruktionen
- Übersicht an Pseudoinstruktionen

School of Computation, Information and Technology
Technische Universität München

Übung 02: RISC-V Assembly

Einführung in die Rechnerarchitektur

Niklas Ladurner

School of Computation, Information and Technology
Technische Universität München

23. Oktober 2026

